Design Document: Library Distribution, Schema Evolution, and Event Driven Deployment

Versioning and distribution across 25+ services in a monorepo

I would package the shared ingestion/processing logic as an internal versioned library with its own semantic version (MAJOR.MINOR.PATCH), even inside the monorepo. Each service would depend on that package through its pyproject.toml, pinned to an explicit version range. This keeps service boundaries clear and avoids hidden breakage from direct cross imports.

Distribution would happen through CI. When the library changes, the pipeline builds a wheel and publishes it to an internal artifact registry. Services can then upgrade intentionally via automated PRs. Patch or minor releases remain backward compatible while major releases are reserved for breaking changes. This gives monorepo convenience without forcing all 25+ services to move in lockstep.

Schema evolution without breaking consumers

The library should expose a stable canonical domain model and treat external schemas as adapters into that model. Schema evolution is then handled at the ingestion boundary, not throughout the codebase.

For new sensor types, I would add new adapters or validators behind a registry. For renamed columns, I would support field aliases and deprecation windows. For different units, I would normalize values into canonical units during ingestion. Events should also carry a schema version so the library can route them through the right translation logic. Older versions remain supported for a defined period, with warnings or metrics to track usage before removal. Contract tests should verify that old producers still work against new library versions.

CI/CD pipeline

The CI/CD pipeline should be split into library validation, service validation, and deployment

- **On every PR**
  - Run formatting, linting, and type checking
  - Run unit tests for the library and services
  - Run contract or schema compatibility tests
  - Run integration tests with a real queue emulator or in memory transport plus API tests
  - Build the library artifact and Docker images
  - Run security/dependency scans

- **On merge to main**
  - Publish the versioned library artifact
  - Build and push producer/consumer container images
  - Deploy first to a staging environment
  - Run smoke tests and end to end event flow checks
  - Promote to production with rolling or blue/green deployment

This keeps quality gates strong while enabling safe, repeatable releases.

Independently scalable producer and consumer deployment

The producer and consumer should run as separate stateless services, each with its own deployment and autoscaling policy. The producer scales based on incoming write/load patterns, the consumer scales based on queue depth and processing lag.

To avoid processing the same event twice, I would use a real message broker with consumer groups so each event is delivered to only one active consumer in the group. Since most brokers provide at least once delivery, the consumer must be idempotent: every event should carry a unique event ID, and processed IDs (or equivalent dedupe

keys) should be stored or checkpointed before side effects are committed. Acknowledge the message only after successful processing and persistence.

When the consumer falls behind the producer

If the consumer cannot keep up, backlog will accumulate in the queue. I would detect this through metrics such as:

- queue depth
- event age or consumer lag
- processing rate and ingest rate
- retry or DLQ rate
- end to end latency

The first response is usually to scale consumers horizontally. If that is insufficient, I would reduce work per message (batch writes, optimize hot paths, move expensive enrichment off the critical path). I would also introduce backpressure controls: retry limits, dead letter queues for poison messages, and alerting when lag crosses thresholds. If needed, the producer can shed non critical traffic or downgrade verbosity. The goal is to keep the system stable, visible, and recoverable rather than letting silent backlog growth become an outage.